Labs

Lab 1: Create a Secure CI/CD Pipeline for a Spring Boot Application

Implement a complete CI/CD workflow for a Java-based Spring Boot application hosted on GitHub. The pipeline must ensure code quality, security, and automate the entire process from build to container image deployment. Follow the steps below:

1. **Trigger Conditions**
    - The workflow must run automatically on every push or pull request to the `main` branch.
2. **Linting and Code Validation**
    - Integrate a Java linter (e.g., GitHub Super-Linter) to check the code style and syntax.
    - Configure the linter to validate only Java code.
3. **Security Scanning**
    - Include a job that scans for exposed secrets using GitGuardian.
    - Include a job that performs static code analysis using Semgrep.
    - Add a container image vulnerability scan using Trivy before pushing to Docker Hub.
4. **Java Build Matrix**
    - Use GitHub Actions' matrix strategy to build the application with two Java versions (e.g., Java 21 and 22).
    - Use Maven to build and test the project.
    - Upload the generated JAR as an artifact.
5. **Docker Image Build and Deployment**
    - Use the JAR built with Java 21 to build a Docker image.
    - Tag the image appropriately and run a vulnerability scan with Trivy.
    - Log in to Docker Hub using secrets and push the image to your Docker repository.
6. **Secrets Management**
    - All sensitive credentials (GitHub tokens, Docker credentials, API keys) must be stored as GitHub repository secrets.

The solution can be found here.

Questions

1. How does the DevOps approach differ from traditional models such as Waterfall and Agile? In what ways does DevOps improve automation in the software development lifecycle?
2. What are the key benefits of DevOps automation, and which tools are commonly used to support it? What is the role of Jenkins in DevOps pipelines?
3. What are the seven C's of DevOps, and how do they contribute to the success of DevOps practices? Does DevOps require only technical implementation, or does it also involve organizational change?
4. What are the differences between continuous integration, continuous delivery, and continuous deployment?
5. What are DORA metrics, and how can an organization improve its performance based on these metrics over time?
6. What deployment strategies are used to improve DORA metrics, and how do they operate?
7. How does DevSecOps incorporate security into the CI/CD pipeline, and what are its main advantages? What challenges are commonly faced when adopting DevSecOps in an organization?
8. What is GitHub Actions and how does it work? What are five widely used CI/CD tools in the industry, and what are their key characteristics?
9. How do Infrastructure as Code (IaC) principles enhance DevOps practices?
10. What is configuration drift, how does it relate to configuration as code, and what are the main tools used to address it?
11. What are the characteristics of monolithic architectures, and what are their main advantages and disadvantages?
12. Explain the *layered architecture* pattern and its key advantages and disadvantages.
13. What is the *clean architecture*, and how does it improve the *layered architecture*?
14. Describe the concept of a modular monolith and its benefits over other monolithic architectures.
15. Explain the *database per service pattern* in microservices architecture. What are the benefits and challenges of this approach?
16. What challenges arise when transitioning from a monolithic architecture to microservices? How does the *Strangler Pattern* relates to these challenges?
17. Discuss the key features and challenges of the *microservices architecture*.
18. Describe the concept of *scale cube* and its three dimensions.